

Reconhecimento de Padrões Lógicos com Redes Neurais MLP e Backpropagation

Pedro Neto, Guilherme Kauã

Instituto Federal de Educação, Ciência e Tecnologia de Brasília (IFB) — Campus Brasília

Resumo. Este trabalho apresenta a implementação de uma rede Perceptron Multicamadas (MLP) com Backpropagation para resolver a função lógica XOR, problema não linearmente separável. A rede foi construída do zero em Python (NumPy), sem frameworks de aprendizado. São realizados experimentos que avaliam a influência do número de neurônios na camada oculta, da taxa de aprendizagem e da inicialização dos pesos sobre a convergência. Os resultados mostram que todos os hiperparâmetros testados afetam significativamente a velocidade de convergência.

Palavras-chave. Redes Neurais Artificiais, MLP, Backpropagation, XOR, funções lógicas.

I. INTRODUÇÃO

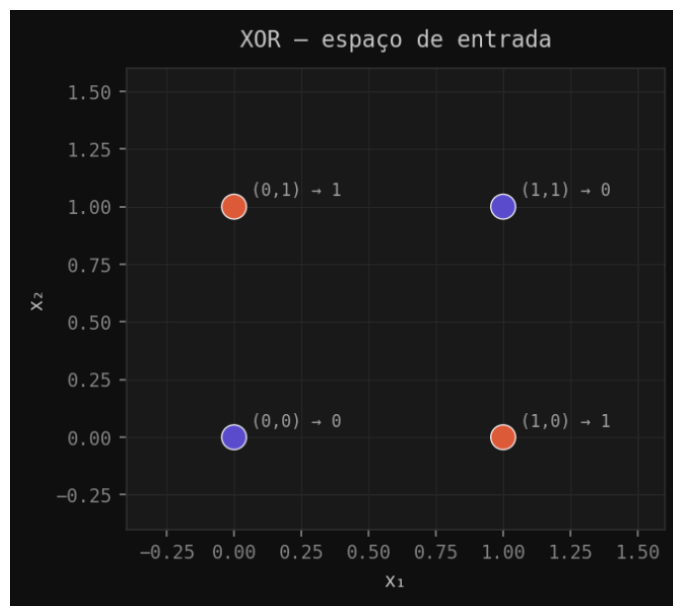
Redes neurais do tipo Perceptron de camada única são incapazes de resolver problemas não linearmente separáveis, como a função lógica XOR. As redes MLP superam essa limitação ao combinar múltiplas camadas com ativações não lineares, treinadas pelo algoritmo de Backpropagation ^[1, 2]. Este trabalho descreve a implementação e os experimentos com uma MLP de uma camada oculta para o XOR, avaliando a influência dos principais hiperparâmetros.

II. FUNDAMENTAÇÃO TEÓRICA

A. O problema XOR

A função XOR retorna 1 quando exatamente uma das entradas binárias é 1. Seus quatro padrões (00→0, 01→1, 10→1, 11→0) não podem ser separados por uma única reta, como ilustrado na Figura 1, exigindo uma fronteira de decisão não linear.

Fig. 1. Espaço de entrada do XOR. Nenhuma reta separa as classes 0 (roxo) e 1 (laranja).

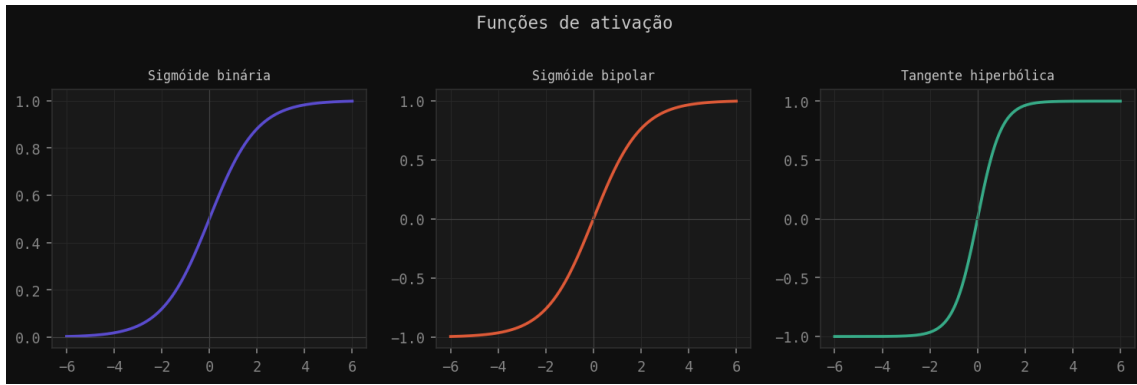


Fonte: Elaborado pelos autores

B. Funções de ativação

Foram implementadas três funções de ativação (Figura 2): a *sigmóide binária* $f(x) = 1/(1+e^{-x})$ com saída em (0,1); a *sigmóide bipolar* $f(x) = 2/(1+e^{-x})-1$ com saída em (-1,1); e a *tangente hiperbólica*, equivalente à bipolar. Todos os experimentos utilizaram a sigmóide bipolar.

Fig. 2. Funções de ativação implementadas: sigmóide binária, bipolar e tangente hiperbólica.



Fonte: Elaborado pelos autores.

C. Backpropagation

A rede possui 2 entradas, uma camada oculta de tamanho variável e 1 saída. Pesos e bias são inicializados em $[-0,5; 0,5]$. Na fase *forward*, cada camada computa a soma ponderada seguida da ativação. Na fase *backward*, o erro é retropropagado camada a camada pela regra delta generalizada ($\Delta w = \eta \cdot \delta \cdot a$). A rede converge quando todas as saídas estão a menos de 0,1 dos valores desejados, com limite de 10.000 épocas.

III. MATERIAIS E MÉTODOS

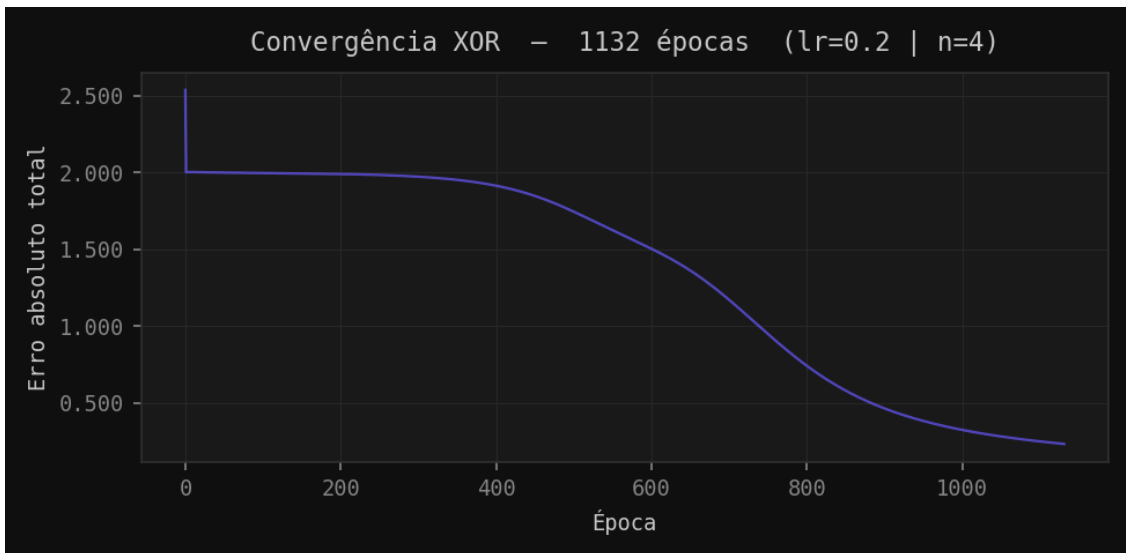
A implementação usa exclusivamente NumPy e Matplotlib, organizada em *mlp.py* (classe MLP) e *experimentos.py* (experimentos). Configuração base: 4 neurônios ocultos, $lr = 0,2$, sigmóide bipolar, $seed = 42$. Experimento A: variação de neurônios (2–5), $lr = 0,5$. Experimento B: variação da taxa (0,1–0,5), $n = 4$. Experimento C: variação de seeds (0–4), $n = 4$, $lr = 0,2$. Por fim, a mesma arquitetura foi aplicada às portas AND, OR, NAND e NOR.

IV. RESULTADOS

A. Experimento principal

Com $lr = 0,2$, $n = 4$ e $seed = 42$, a rede convergiu em **1.132 épocas**. Saídas finais: (0,0)→0,012; (0,1)→0,907; (1,0)→0,900; (1,1)→0,025 — confirmando o aprendizado correto do XOR (Figura 3).

Fig. 3. Convergência do experimento principal ($n=4$, $lr=0,2$, $seed=42$): 1.132 épocas.

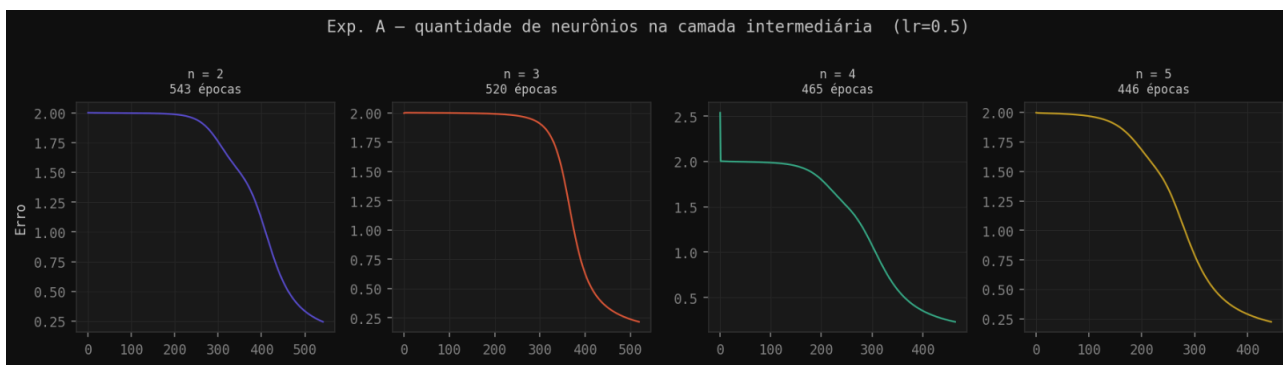


Fonte: Elaborado pelos autores.

B. Experimento A — número de neurônios

Com $lr = 0,5$ e $seed = 42$: 543 ($n=2$), 520 ($n=3$), 465 ($n=4$) e 446 ($n=5$) épocas (Figura 4). O ganho é decrescente — entre 4 e 5 neurônios a diferença é pequena, indicando que o XOR já está bem representado com 4 unidades ocultas.

Fig. 4. Convergência para 2, 3, 4 e 5 neurônios ($lr=0,5$, $seed=42$).

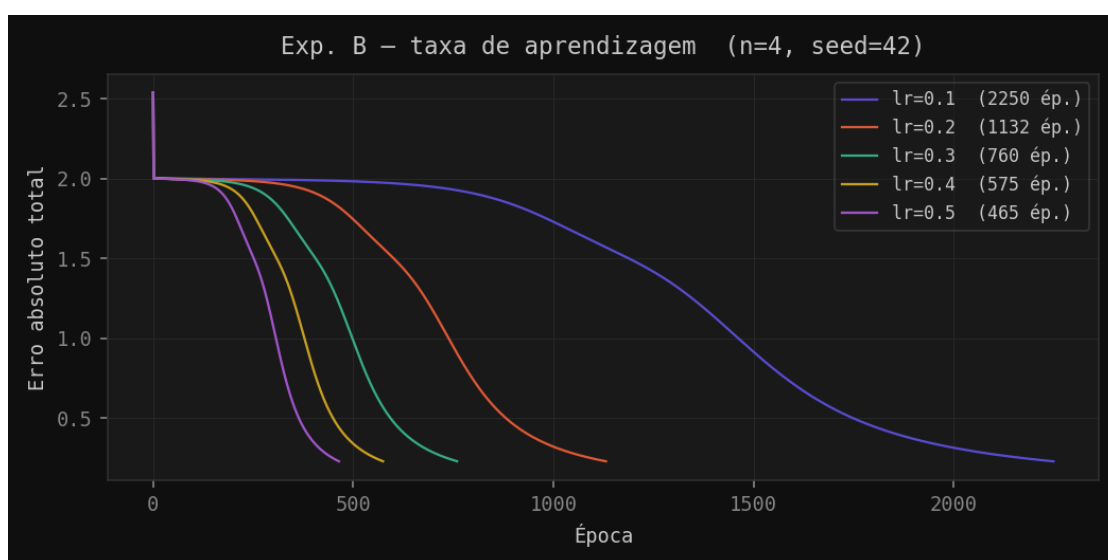


Fonte: Elaborado pelos autores.

C. Experimento B — taxa de aprendizagem

Mantendo $n = 4$ e $\text{seed} = 42$: 2.250 ($\text{lr}=0,1$), 1.132 ($\text{lr}=0,2$), 760 ($\text{lr}=0,3$), 575 ($\text{lr}=0,4$) e 465 ($\text{lr}=0,5$) épocas (Figura 5). Taxas maiores aceleram a convergência dentro da faixa testada.

Fig. 5. Convergência para taxas de aprendizagem entre 0,1 e 0,5 ($n=4$, $\text{seed}=42$)

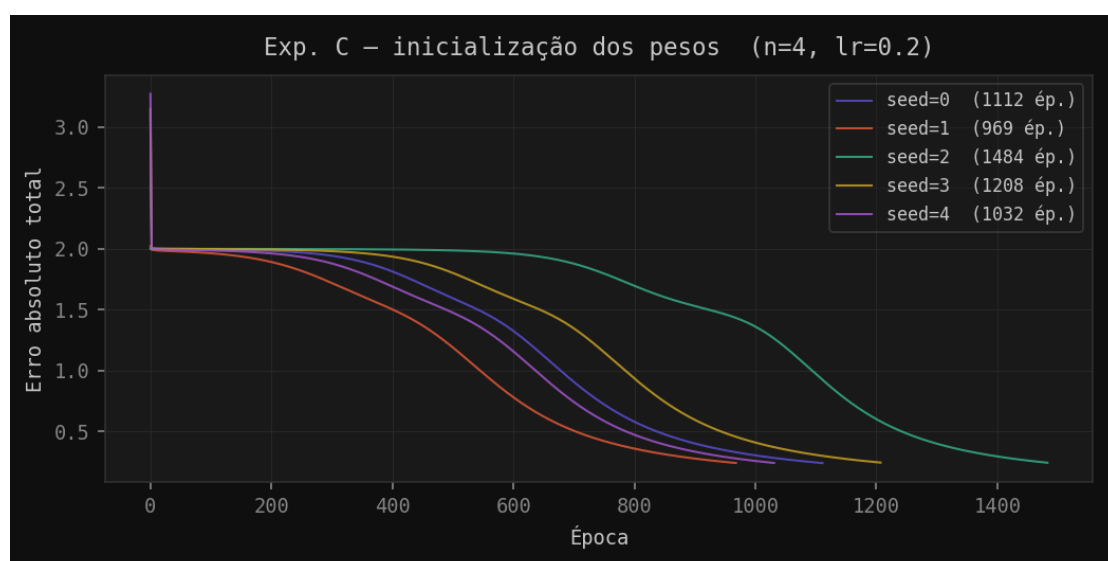


Fonte: Elaborado pelos autores..

D. Experimento C — inicialização dos pesos

Com $n = 4$ e $\text{lr} = 0,2$: 1.112, 969, 1.484, 1.208 e 1.032 épocas para seeds 0–4 (Figura 6). A variação de até 53% entre o melhor ($\text{seed}=1$) e o pior caso ($\text{seed}=2$) evidencia influência significativa da inicialização.

Fig. 6. Convergência para cinco inicializações aleatórias ($n=4$, $\text{lr}=0,2$).

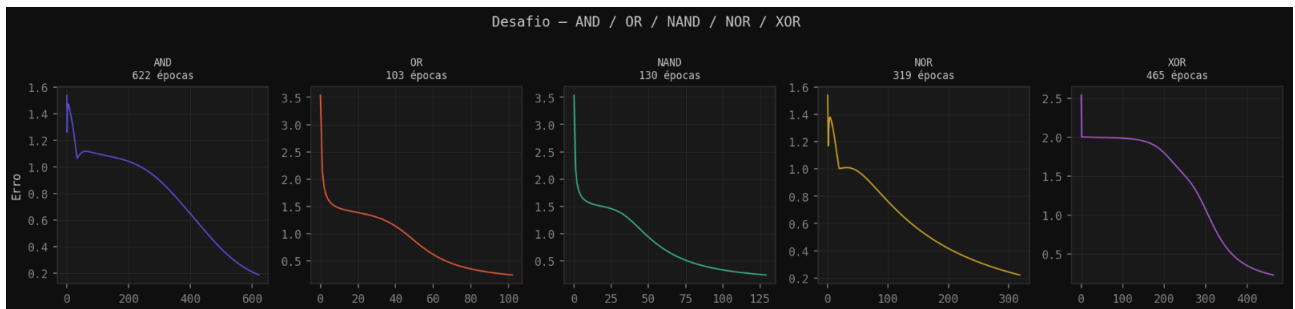


Fonte: Elaborado pelos autores.

E. Funções lógicas AND, OR, NAND, NOR e XOR

Com $n = 4$, $lr = 0,5$ e $seed = 42$: AND (622), OR (103), NAND (130), NOR (319) e XOR (465) épocas (Figura 7). OR e NAND convergem mais rápido porque a saída positiva está associada à maioria das entradas; AND e XOR exigem mais épocas pela menor frequência ou não-linearidade da saída 1.

Fig. 7. Convergência para AND, OR, NAND, NOR e XOR ($n=4$, $lr=0,5$, $seed=42$)



Fonte: Elaborado pelos autores..

V. DISCUSSÃO

Os experimentos confirmam que uma camada oculta com ativação não linear é suficiente para resolver o XOR. Os três hiperparâmetros avaliados afetam a velocidade de convergência sem impedir a solução dentro das faixas testadas. A diferença em relação aos 3.387 ciclos do artigo de referência [3] (obtidos com $lr = 0,2$ e $n = 4$) deve-se ao modo de atualização: o artigo usa treinamento padrão a padrão (online), enquanto nossa implementação atualiza os pesos por época completa (batch), resultando em gradientes mais estáveis e convergência mais rápida.

VI. CONCLUSÃO

Este trabalho demonstrou que uma MLP com uma camada oculta treinada por Backpropagation aprende a função XOR e outras portas lógicas. A implementação do zero permitiu observar diretamente os passos forward e backward, bem como a influência dos hiperparâmetros no treinamento. Os resultados evidenciam que a dificuldade de aprendizado está relacionada à complexidade da fronteira de decisão de cada problema.

MATERIAL COMPLEMENTAR

O código-fonte completo da implementação (incluindo a classe MLP, os scripts de experimentos e as figuras geradas), bem como uma versão organizada dos arquivos para consulta, estão disponíveis nos links abaixo:

Repositório (código-fonte):

<https://github.com/netod3v/mlp-xor-backpropagation/tree/main>

Pasta com arquivos e resultados:

<https://drive.google.com/drive/folders/10NBpG56JI10GrRszfZG1st-DtjdQFKQc?usp=sharing>

REFERÊNCIAS

- [1] A. Braga, A. Carvalho e T. Ludermir. *Redes Neurais Artificiais: teoria e aplicações*. LTC, 2000.
- [2] L. Fausett. *Fundamentals of Neural Networks*. Prentice-Hall, 1994.
- [3] F. H. M. Oliveira e P. H. C. Braga. "Implementação de funções lógicas utilizando retropropagação do erro". *Redes Neurais Artificiais*, UFU, 2011.
- [4] D. E. Rumelhart, G. E. Hinton e R. J. Williams. "Learning representations by back-propagating errors". *Nature*, 1986.